

Подготовка к экзамену по C++ (ИТМО)

Тема 3: Указатели (Сырые, `void*`, на функции и методы)

Hardcore Theory Edition

15 июня 2026 г.

1 Анатомия указателя и арифметика

Указатель — это переменная, которая хранит адрес другой переменной в оперативной памяти. На 64-битной системе любой указатель (хоть `char*`, хоть `double*`) весит строго **8 байт**.

1.1 Арифметика указателей

Тип указателя нужен компилятору для того, чтобы понимать две вещи:

1. Сколько байт прочитать при разыменовании (`*ptr`).
2. На сколько байт сдвинуться при арифметике (`ptr + 1`).

Если `int* ptr = 0x1000`, то выражение `ptr + 1` даст адрес `0x1004` (так как `sizeof(int)` = 4). Компилятор неявно умножает прибавляемое число на размер типа! Разность двух указателей (`ptr2 - ptr1`) возвращает не разницу в байтах, а **количество элементов** между ними (тип `ptrdiff_t`).

Суть на пальцах (Жизненная аналогия)

Указатель — это адрес дома (например, "ул. Ленина, 10"). Тип указателя (`int`) — это чертеж. Когда вы говорите "покажи дом номер 10" чертеж подсказывает, что нужно осмотреть 4 этажа. Когда вы говорите "иди к следующему дому" (`ptr + 1`), чертеж подсказывает, что нужно отступить ровно на 4 этажа вперед, чтобы не попасть в чужую спальню.

2 `void*` (Бестиповый указатель)

`void*` — это чистый, "голый" адрес в памяти. У него нет типа.

- В `void*` можно неявно положить любой указатель.
- `void*` **нельзя разыменовывать** (компилятор не знает, сколько байт читать).
- К `void*` **нельзя применять арифметику** (компилятор не знает, на сколько шагать, так как `sizeof(void)` не существует).
- Чтобы с ним работать, его нужно явно скастовать (например, через `static_cast<int*>`).

3 Указатели на функции

Функции не лежат на стеке или в куче, они лежат в специальном сегменте памяти (сегмент кода / Text segment). И у них тоже есть адреса! Мы можем сохранить адрес функции в переменную и вызвать её позже. Это основа для *коллбеков* (*callbacks*).

Синтаксис (надо выучить наизусть):

ВозвращаемыйТип (*ИмяУказателя) (ТипыАргументов);

Пример: указатель на функцию, принимающую два `int` и возвращающую `int`:

```
1 int add(int a, int b) { return a + b; }
2
3 // Создаем указатель
4 int (*func_ptr)(int, int) = &add;
5
6 // Вызываем
7 int result = func_ptr(5, 3); // Вернет 8
```

4 Указатели на члены класса (Pointers to Members)

Это самая дикая часть плюсов. Обычный указатель указывает на конкретный адрес в оперативной памяти. **Указатель на член класса — это не адрес!** Это смещение (offset) относительно начала объекта.

Вы не можете разыменовать указатель на метод класса сам по себе. Вам нужен **конкретный объект**, к которому этот метод или поле будет применено.

```
1 struct Robot {
2     int hp;
3     void attack() { std::cout << "Pew pew!"; }
4 };
5
6 int main() {
7     // Указатель на поле hp в любом объекте класса Robot
8     int Robot::*hp_ptr = &Robot::hp;
9
10    // Указатель на метод attack
11    void (Robot::*action_ptr)() = &Robot::attack;
12
13    Robot r1;
14    r1.hp = 100;
15
16    // Специальные операторы: .* (если объект по значению/ссылке) и ->* (если указатель)
17    std::cout << r1.*hp_ptr; // Выведет 100
18    (r1.*action_ptr)();      // Вызовет r1.attack()
19 }
```

Суть на пальцах (Жизненная аналогия)

Указатель на метод класса — это инструкция "нажми на 3-ю кнопку слева". Вы не можете просто выполнить эту инструкцию висая в воздухе. Вам нужен **конкретный пульт управления** (объект). Вы берете конкретный пульт (объект `r1`), применяете к нему инструкцию (оператор `.*`) и нажимаете 3-ю кнопку.